



Dashboard de telemetria

Telemática

Nombre Completo

Juan Andrés Young Hoyos

Ismael García Ceballos

Samuel López Gómez



20 de Septiembre de 2026

Índice

1	Introducción y descripción del problema	2
2	Arquitectura del sistema	2
2.1	Nodos de telemetría	3
2.2	Servidor central	3
2.3	Cliente operador	3
2.4	Servicio web	3
3	Diseño y especificación del protocolo TELEP/2.0	3
3.1	Formato e identidad	3
3.2	Tipos de mensaje	3
3.3	Alertas	4
3.4	Códigos de error	4
3.5	Ejemplo de intercambio	4
3.6	Por qué UDP para la telemetría y TCP para lo demás	5
4	Implementación de sockets TCP y UDP	5
4.1	Servidor	6
4.2	Clientes	6
5	Concurrencia y programación de clientes y servidor	7
6	Despliegue en nube, DNS y Docker	8
6.1	Instancia y reglas de acceso	8
6.2	DNS	9
6.3	Docker	9
7	Análisis de tráfico con Wireshark	10
8	Pruebas y resultados	10
8.1	Mensajes transmitidos, recibidos y perdidos	11
9	Problemas encontrados y soluciones implementadas	12
10	Conclusiones	12
A	Evidencias individuales	13
A.1	Juan Andrés Young	13
A.2	Samuel López	16
A.3	Ismael García	19
B	Video de sustentación	22
C	Referencias y archivos	22

1 | Introducción y descripción del problema

Una organización que administra infraestructura inteligente necesita monitorear instalaciones distribuidas geográficamente. Cada instalación tiene dispositivos IoT que reportan variables como temperatura, humedad, consumo energético, vibración y estado operativo, y esa información debe llegar a un servidor central en la nube desde donde los operadores puedan consultar el estado de los dispositivos y recibir alertas cuando se detecten condiciones anómalas.

En este proyecto simulamos esa infraestructura con aplicaciones distribuidas que se comunican por Internet: nodos en Python que envían mediciones, un servidor en C desplegado en Docker sobre una instancia EC2, y un cliente operador. La comunicación usa un protocolo de aplicación propio, TELEP, sobre sockets TCP y UDP, y el servidor localiza por DNS. El informe describe el protocolo, la implementación de los sockets, la concurrencia, el despliegue, el análisis del tráfico con Wireshark y las pruebas realizadas.

2 | Arquitectura del sistema

El sistema tiene tres componentes principales y uno complementario, como muestra la Figura 2.1: los nodos de telemetría y los clientes operador (Python) corren en los computadores de los integrantes; el servidor central (C) corre en la nube; el navegador consulta la página de estado. Los tres puertos del servidor se resumen en la Tabla 2.1.

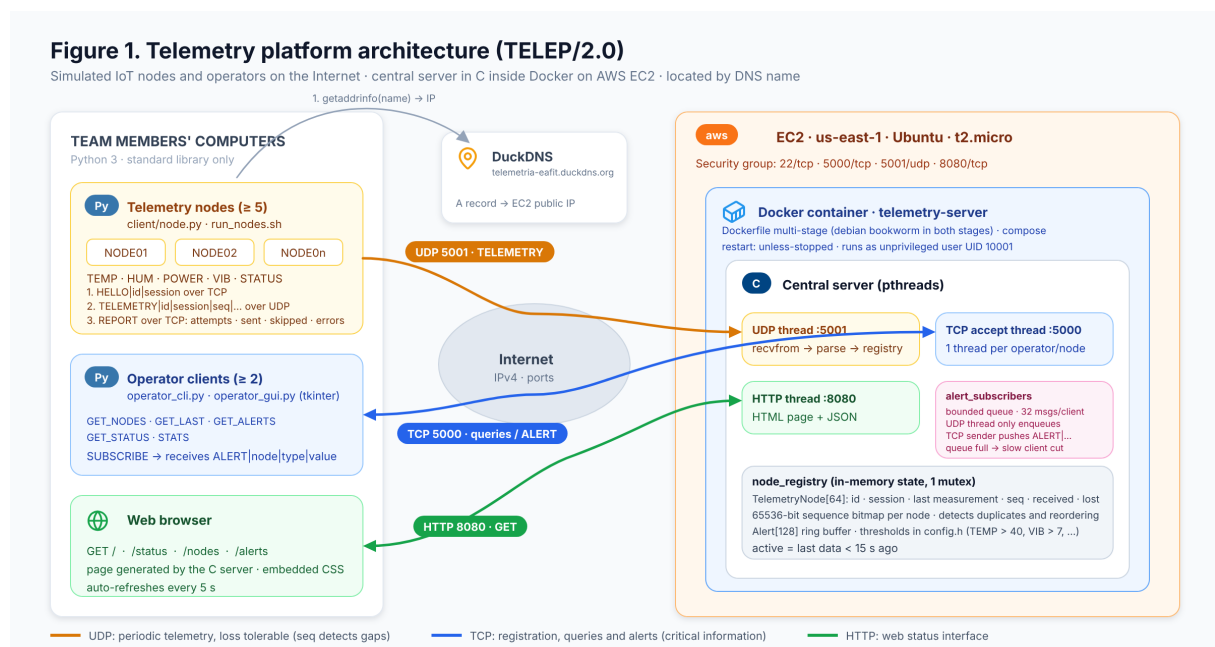


Figura 2.1: Arquitectura general. Los nodos envían telemetría por UDP al puerto 5001; el registro de nodos, las consultas de los operadores y las alertas van por TCP al puerto 5000; el navegador lee la página de estado por HTTP en el 8080. Los clientes localizan el servidor por su nombre DNS, nunca por una dirección IP fija.

Cuadro 2.1: Puertos del servidor y su función.

Puerto	Transporte	Quién lo usa	Para qué
5000	TCP	nodos y operadores	registro (HELLO), consultas, reportes y alertas
5001	UDP	nodos	telemetría periódica
8080	TCP (HTTP)	navegador	página de estado y recursos JSON

2.1 | Nodos de telemetría

Cada nodo (`client/node.py`) tiene un identificador único, localiza el servidor por DNS y se registra por TCP con HELLO. Después envía por UDP, cada pocos segundos, un mensaje **TELEMETRY** con cinco variables simuladas: temperatura, humedad, consumo, vibración y estado. Si el servidor no responde, reintenta en lugar de terminar.

El nodo también tiene tres opciones que usamos en las pruebas: `--spike TEMP=45` fija la temperatura en 45 grados para que el servidor genere una alerta, `--fail-status` hace que el nodo reporte **STATUS=FAIL**, y `--drop 25` deja de enviar el 25 % de los datagramas para comprobar que el servidor cuenta bien los que se pierden.

2.2 | Servidor central

El servidor (`server/`) mantiene en memoria el registro de nodos con su última medición, contadores por sesión y un historial de alertas. Atiende a la vez la telemetría UDP, varias conexiones TCP de operadores y nodos, y las peticiones HTTP, con un hilo por servicio y un hilo por conexión TCP (Figura 5.1). Detecta valores anómalos comparándolos con umbrales, genera la alerta al cruzar el umbral y la entrega a los operadores suscritos. Un mensaje malformado o un cliente que se desconecta de golpe nunca terminan el proceso.

2.3 | Cliente operador

El cliente operador (`client/operator_cli.py`) es un programa de consola que usa el módulo `client/telep.py` para conectarse al servidor por TCP. Permite listar los nodos activos e inactivos, ver las últimas mediciones de todos o de un nodo, consultar las alertas, el estado general y las estadísticas de entrega. Al conectarse se suscribe, así que las alertas nuevas le llegan sin tener que pedir las.

2.4 | Servicio web

El propio servidor sirve en el puerto 8080 una página HTML y tres recursos JSON (`/status`, `/nodes`, `/alerts`) con la misma información que las consultas TELEP. Es solo lectura; no reemplaza al protocolo propio.

3 | Diseño y especificación del protocolo TELEP/2.0

TELEP (*Telemetry Exchange Protocol*) es un protocolo de texto. Cada mensaje es una línea ASCII, con campos separados por |. Las mediciones viajan en un único campo como pares **NOMBRE=valor** separados por ;. Se eligió texto sobre binario porque se puede leer directamente en Wireshark y probar con herramientas como nc, lo que facilita depurar y explicar el protocolo. La especificación completa está en `docs/PROTOCOL.md` del repositorio.

3.1 | Formato e identidad

El identificador de nodo es una cadena de 1 a 15 caracteres alfanuméricos, guion o guion bajo. Cada ejecución de un nodo genera una sesión de 32 caracteres hexadecimales que acompaña a sus mensajes; así el servidor distingue un nodo que se reinició de otro proceso que intenta usar el mismo identificador (en ese caso responde `ERR|105|ID_IN_USE`). La secuencia UDP va de 0 a 65535 y el servidor conserva en un mapa de bits qué secuencias recibió durante toda la sesión, lo que permite contar únicos, duplicados y reordenados.

3.2 | Tipos de mensaje

La Tabla 3.1 resume los mensajes y su transporte.

Las variables admitidas son **TEMP** (−100 a 200), **HUM** (0 a 100), **POWER** (0 a 1 000 000), **VIB** (0 a 1000) y **STATUS** (OK o FAIL). Un datagrama debe traer al menos tres variables distintas; si trae campos vacíos, nombres desconocidos, duplicados o números no finitos se descarta sin respuesta y se contabiliza como inválido.

Cuadro 3.1: Mensajes de TELEP/2.0.

Mensaje	Transporte	Respuesta
HELLO id session	TCP	OK REGISTERED
TELEMETRY id session seq mediciones	UDP	ninguna (mejor esfuerzo)
REPORT id session attempts sent omitted send_errors final	TCP	OK REPORTED
GET_STATUS	TCP	OK clave=valor;...
GET_NODES	TCP	OK n, luego n líneas id ACTIVE edad (o INACTIVE) y END
GET_LAST id	TCP	OK id epoch mediciones o OK id 0 NO_DATA
GET_ALERTS	TCP	OK n, luego n líneas epoch id tipo valor y END
STATS o STATS id	TCP	OK n, luego n líneas id session clave=valor;... y END
SUBSCRIBE	TCP	OK SUBSCRIBED; después el servidor envía ALERT id tipo valor
BYE	TCP	OK BYE y cierre de la conexión

3.3 | Alertas

Cuando una medición supera su umbral ($TEMP > 40$, $HUM > 85$, $POWER > 500$, $VIB > 7$, o $STATUS=FAIL$) el servidor genera una alerta del tipo `TEMP_HIGH`, `HUM_HIGH`, `POWER_HIGH`, `VIB_HIGH` o `STATUS_FAIL`. La alerta se genera una sola vez al entrar en la anomalía; los datagramas siguientes con el valor todavía alto no la repiten. Se guarda en un historial de 128 entradas consultable con `GET_ALERTS` y, para los operadores que enviaron `SUBSCRIBE`, se empuja de inmediato por su conexión TCP como `ALERT|NODE01|TEMP_HIGH|45.00`. Como esa línea puede llegar entre una petición y su respuesta, los clientes la reconocen por el prefijo y la muestran aparte.

3.4 | Códigos de error

Los errores tienen la forma `ERR|código|nombre`: 100 `BAD_FORMAT`, 101 `UNKNOWN_CMD`, 102 `UNKNOWN_NODE`, 103 `TOO_LONG`, 104 `SERVER_FULL` y 105 `ID_IN_USE`. Un error nunca cierra la conexión: el cliente puede seguir enviando comandos, cosa que se comprueba en la sección de pruebas.

3.5 | Ejemplo de intercambio

La Figura 3.1 muestra una sesión completa. Primero un nodo se registra, envía dos mediciones por UDP (la segunda con la temperatura por encima del umbral) y reporta cuántos datagramas envió. Después un operador se suscribe, consulta los nodos, recibe la alerta que generó la segunda medición, pide un nodo que no existe y cierra. Las líneas con `>` las envía el cliente y las de `<` el servidor; la sesión de 32 caracteres se muestra abreviada como `9f3c...a2e1`.

```

> HELLO|NODE01|9f3c...a2e1
< OK|REGISTERED
  (UDP) TELEMETRY|NODE01|9f3c...a2e1|0|TEMP=24.8;HUM=60;POWER=120;VIB=0.5;STATUS=OK
  (UDP) TELEMETRY|NODE01|9f3c...a2e1|1|TEMP=45.1;HUM=60;POWER=120;VIB=0.5;STATUS=OK
> REPORT|NODE01|9f3c...a2e1|2|2|0|0|1
< OK|REPORTED

> SUBSCRIBE
< OK|SUBSCRIBED
> GET_NODES
< OK|1
< NODE01|ACTIVE|1
< END
< ALERT|NODE01|TEMP_HIGH|45.10
> GET_LAST|NODE99
< ERR|102|UNKNOWN_NODE
> BYE
< OK|BYE

```

Figura 3.1: Sesión de ejemplo de TELEP/2.0: registro y telemetría de un nodo, consulta de un operador, alerta y error.

3.6 | Por qué UDP para la telemetría y TCP para lo demás

Usamos UDP para la telemetría porque es información que se repite: cada nodo envía una medición nueva cada pocos segundos, así que si un datagrama se pierde la siguiente lo reemplaza y no hace falta retransmitirlo. UDP no establece conexión ni envía acuses, por lo que cada medición cuesta un solo datagrama de 8 bytes de encabezado, y el servidor no tiene que mantener una conexión abierta por cada nodo. Para saber cuánto se perdió, cada mensaje lleva un número de secuencia y el servidor cuenta los que faltan.

Usamos TCP para el registro del nodo, las consultas del operador, los reportes de conteo y las alertas porque ahí sí importa que cada mensaje llegue completo, en orden y con confirmación: una consulta debe recibir su respuesta y una alerta no puede perderse. TCP garantiza eso con el establecimiento de conexión, los números de secuencia y las retransmisiones. Por la misma razón las alertas se envían al operador por su conexión TCP ya abierta, y no por UDP.

Cuadro 3.2: Transporte elegido para cada tipo de mensaje.

Mensajes	Transporte	Motivo
TELEMETRY	UDP 5001	periódica, tolera pérdida, sin conexión por nodo
HELLO, REPORT	TCP 5000	el nodo necesita confirmación
GET_*, STATS	TCP 5000	respuestas completas y en orden
ALERT	TCP 5000	información crítica, no puede perderse

4 | Implementación de sockets TCP y UDP

Las operaciones de sockets están escritas de forma explícita en el código, sin bibliotecas intermedias. La Tabla 4.1 indica dónde está cada una.

Cuadro 4.1: Operaciones de sockets y su ubicación en el código.

Operación	Servidor (C)	Clientes (Python)
Creación	<code>socket()</code> en <code>socket_helpers.c</code>	<code>socket.socket()</code> en <code>telep.py</code> y <code>node.py</code>
Conexión	—	<code>connect()</code> en <code>telep.py</code>
Escucha y aceptación	<code>bind()</code> y <code>listen()</code> en <code>socket_helpers.c</code> ; <code>accept()</code> en <code>tcp_command_server.c</code>	—
Envío y recepción	<code>send()</code> , <code>recv()</code> y <code>recvfrom()</code>	<code>sendall()</code> , <code>readline()</code> y <code>sendto()</code>
Cierre	<code>close()</code> en <code>tcp_command_server.c</code>	<code>close()</code> en <code>telep.py</code> y <code>node.py</code>
Errores	<code>perror()</code> tras cada llamada; SIGPIPE ignorado	<code>except OSError</code> con reintento

4.1 | Servidor

Las llamadas de sockets del servidor están en `socket_helpers.c` para no repetirlas en los tres servicios: una función crea el socket TCP, lo vincula a todas las interfaces y lo pone a escuchar; otra crea y vincula el socket UDP; otras dos envían un búfer completo y leen una línea hasta el salto de línea, con límite de tamaño y de tiempo. Sobre ellas, `tcp_command_server.c` acepta conexiones con `accept()` y `udp_telemetry_server.c` recibe datagramas con `recvfrom()`. La Figura 4.1 muestra la creación del socket de escucha.

```
int create_tcp_listener(int port) {
    int listener_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (listener_fd < 0) { perror("socket(TCP)"); return -1; }

    int reuse_address = 1;
    setsockopt(listener_fd, SOL_SOCKET, SO_REUSEADDR,
               &reuse_address, sizeof reuse_address);

    if (bind_to_port(listener_fd, port) < 0) return -1;
    if (listen(listener_fd, TCP_BACKLOG) < 0) {
        perror("listen");
        close(listener_fd);
        return -1;
    }
    return listener_fd;
}
```

Figura 4.1: Creación del socket TCP de escucha en `server/socket_helpers.c`: `socket()`, `bind()` y `listen()`, con `perror()` en cada error.

4.2 | Clientes

`telep.py` resuelve el nombre del servidor con `socket.getaddrinfo`, abre la conexión TCP, codifica los comandos, lee las respuestas línea a línea y separa las alertas empujadas. `node.py` usa esa conexión para HELLO y REPORT, y un socket `SOCK_DGRAM` propio para `sendto` de la telemetría. Si el servidor no responde al registrarse, el nodo reintenta cada pocos segundos en lugar de terminar.

```

Sep 20 14:47
.../EAFIT/telemetria/proyecto01

javyoungpc
javyoungpc
2024-09-20T14:47:21-05:00
Server: 127.0.0.53
Address: 127.0.0.53#53

Non-authoritative answer:
Name: telemetria-eafit.duckdns.org
Address: 100.58.193.142

ip-172-31-84-220
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
7771d65f34be telemetry-server:latest "/telemetry_server" About an hour ago Up 2 minutes 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:5001->5001/udp, [::]:5001->5001/udp
5000/tcp -> 0.0.0.0:5000
5001/udp -> [::]:5001
5001/udp -> 0.0.0.0:5001
5001/udp -> [::]:5001
8080/tcp -> 0.0.0.0:8080
8080/tcp -> [::]:8080
uid=10001 gid=10001 groups=10001

```

Figura 4.2: Nodo de telemetría ejecutándose contra el servidor en la nube desde el computador de un integrante, con la identificación del equipo en la misma terminal.

5 | Concurrencia y programación de clientes y servidor

El servidor usa hilos POSIX (Figura 5.1). `main()` crea los tres sockets y lanza un hilo para UDP, uno que acepta conexiones TCP y uno para HTTP. Cada conexión TCP aceptada recibe su propio hilo, que lee comandos hasta `BYE` o hasta que el cliente se desconecta; así varios operadores y nodos se atienden en paralelo sin que uno bloquee a otro. Se eligió esta estrategia frente a la multiplexación con `select/epoll` porque el código de cada cliente queda lineal (leer línea, procesar, responder) y el estado compartido es pequeño.

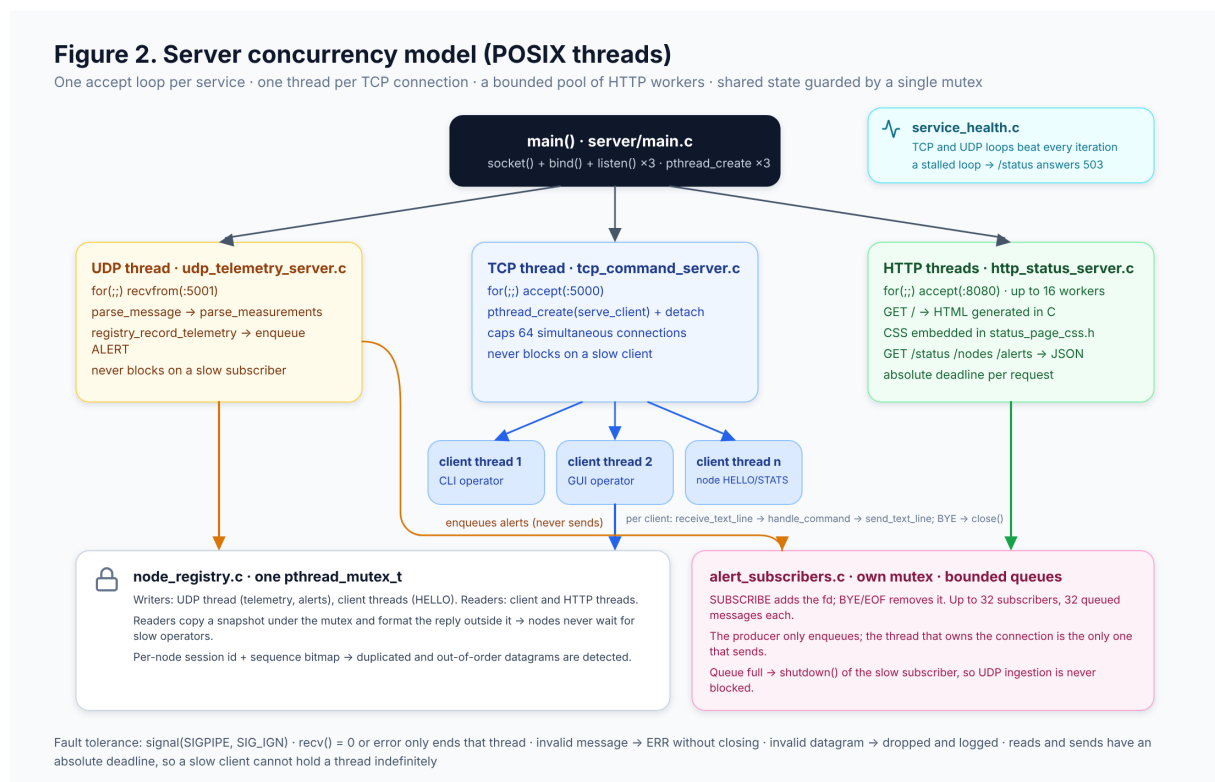


Figura 5.1: Modelo de concurrencia del servidor: un hilo fijo por servicio, un hilo por conexión TCP y un registro compartido protegido por un mutex.

El registro de nodos y alertas está protegido por un único mutex. Los hilos que responden consultas copian una instantánea del registro bajo el mutex y formatean la respuesta fuera de él, de modo que la recepción UDP nunca espera por un operador lento. Cada conexión tiene un único hilo escritor: las alertas para un suscriptor se depositan en una cola acotada de 32 entradas y ese hilo las envía entre respuestas completas, sin intercalar bytes de mensajes distintos; si la cola se llena, el suscriptor se desconecta y puede recuperar el historial con `GET_ALERTS`. La Figura 5.2 sigue el recorrido de una alerta desde el datagrama hasta el

operador.

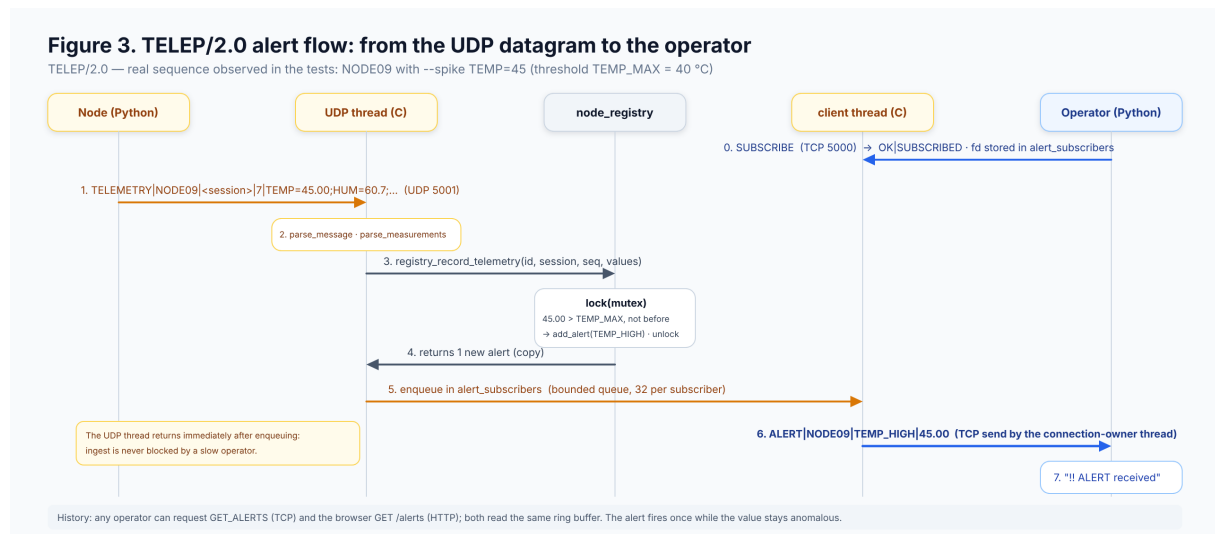


Figura 5.2: Recorrido de una alerta: el datagrama llega al hilo UDP, el registro detecta el cruce de umbral bajo el mutex, y la alerta se entrega a los operadores suscritos por TCP.

La robustez ante fallos se resuelve en tres puntos: SIGPIPE se ignora y todo envío usa MSG_NOSIGNAL, para que un cliente que cierra a mitad de un `send` no mate el proceso; un `recv` que devuelve cero o error solo termina el hilo de ese cliente; y un mensaje inválido produce una respuesta `ERR` sin cerrar la conexión, mientras que un datagrama inválido se descarta y se cuenta. Las peticiones HTTP tienen límites de tamaño y de tiempo, y un máximo de 16 trabajadores, para que una petición incompleta no retenga el servicio.

En el lado Python, el nodo captura `SIGINT` y `SIGTERM` para cerrar ordenadamente y enviar su reporte final; la interfaz gráfica refresca cada tres segundos y reintenta tras un error de red conservando la ventana.

6 | Despliegue en nube, DNS y Docker

6.1 | Instancia y reglas de acceso

El servidor corre en una instancia EC2 `t2.micro` con Ubuntu Server en la región `us-east-1`, creada desde la consola del AWS Academy Learner Lab con el par de llaves que provee el laboratorio. El grupo de seguridad tiene cuatro reglas de entrada (Tabla 6.1). La regla UDP fue la que más cuidado exigió: si se deja como TCP, la página web responde y los nodos “envían”, pero el servidor nunca recibe un datagrama, sin ningún error visible.

Cuadro 6.1: Reglas de entrada del grupo de seguridad `telemetry-sg`.

Tipo	Protocolo	Puerto	Origen
SSH	TCP	22	0.0.0.0/0
Custom TCP	TCP	5000	0.0.0.0/0
Custom UDP	UDP	5001	0.0.0.0/0
Custom TCP	TCP	8080	0.0.0.0/0

telemetry-server	i-02fad2643ab65bf60	Running	t2.micro	Initializing	us-east-1a	ec2-54-172-185-133.co...	54. [REDACTED]
------------------	---------------------	---------	----------	--------------	------------	--------------------------	----------------

i-02fad2643ab65bf60 (telemetry-server)						
Name	Security group rule ID	Port range	Protocol	Source	Security groups	
-	sgr-0b71159894254c96b	8080	TCP	0.0.0.0/0	telemetry-sg ↗	
-	sgr-0709a329fc490aeda	5000	TCP	0.0.0.0/0	telemetry-sg ↗	
-	sgr-065ed62ab34540720	22	TCP	0.0.0.0/0	telemetry-sg ↗	
-	sgr-073c40c6befd9fdce	5001	UDP	0.0.0.0/0	telemetry-sg ↗	

Figura 6.1: Despliegue del servidor en la instancia con `deploy/deploy.sh`: copia del código, construcción de la imagen y arranque del contenedor con los tres puertos publicados.

6.2 | DNS

El código no contiene ninguna dirección IP pública. Los clientes reciben un nombre por `--host` o por la variable `TELEP.SERVER.HOST` y lo resuelven con `getaddrinfo` antes de crear cada socket. El nombre es `telemetria-eafit.duckdns.org`, un subdominio gratuito de DuckDNS, porque Route 53 no está disponible en el Learner Lab.

El laboratorio apaga la instancia al terminar cada sesión de cuatro horas y, al encenderla de nuevo, AWS le asigna otra IP pública. Para que el nombre siga siendo válido, un servicio de `systemd` en la instancia (`deploy/duckdns.service`) ejecuta al arrancar `deploy/duckdns-update.sh`, que consulta la IP pública actual al servicio de metadatos de EC2 y actualiza el registro en DuckDNS con un token guardado únicamente en la instancia. Comprobamos el mecanismo a lo largo de varias sesiones: la IP cambió de 34.226.153.93 a 13.218.65.204 y luego a 3.85.238.127, y los nodos siguieron funcionando con el mismo nombre sin cambiar nada del lado del cliente (Figura 6.2).

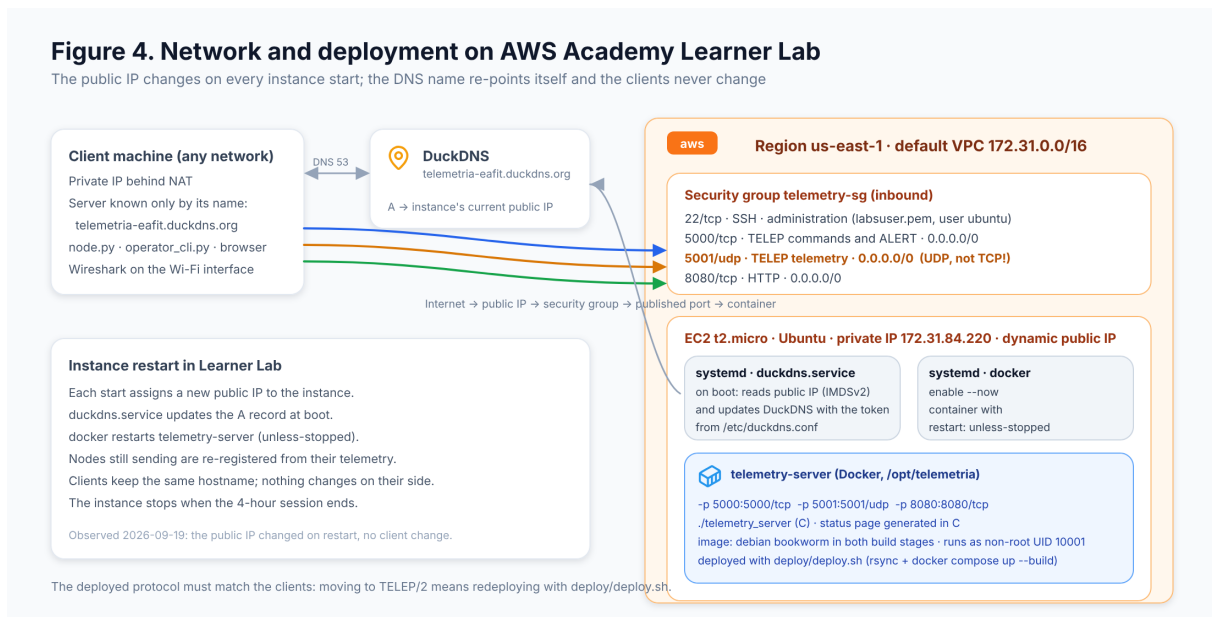


Figura 6.2: Red y despliegue: el cliente resuelve el nombre en DuckDNS, el tráfico entra por el grupo de seguridad a los puertos publicados por Docker, y al reiniciar la instancia el servicio de DuckDNS reapunta el nombre a la nueva IP.

6.3 | Docker

La imagen se construye en dos etapas: una con `gcc` y `make` que compila el servidor con `-Wall -Wextra -Werror`, y una final basada en `debian:bookworm-slim` que solo contiene el binario y se ejecuta con un usuario sin privilegios. `docker-compose.yml` publica `5000:5000/tcp`, `5001:5001/udp` y `8080:8080/tcp` y configura `restart: unless-stopped`, de modo que el contenedor vuelve solo cuando la instancia se

enciende. El sufijo `/udp` en el mapeo del 5001 es imprescindible: sin él, Docker publica el puerto en TCP y la telemetría no entra.

El despliegue se hace con `deploy/deploy.sh` desde el computador de un integrante: copia `server/`, `web/` y los archivos de Docker a `/opt/telemetria` en la instancia con `rsync`, instala el servicio de DuckDNS, construye la imagen en la instancia y recrea el contenedor. Nunca se clona el repositorio en la nube ni se copian credenciales.

7 | Análisis de tráfico con Wireshark

La Figura 7.1 es una captura tomada en el computador de un integrante mientras el operador se conectaba al servidor en la nube. Origen 192.168.1.10 (IP privada del computador) y destino 100.58.193.142 (IP pública de la instancia): capa de red. Puerto 44484 del cliente y 5000 del servidor, protocolo TCP: capa de transporte. Los paquetes 6, 7 y 8 son el establecimiento de la conexión (SYN, SYN, ACK, ACK), y desde el 9 los segmentos PSH, ACK llevan los mensajes TELEP: capa de aplicación. La longitud de cada segmento coincide con el texto enviado, por ejemplo `Len=10` es `SUBSCRIBE` y `Len=14` su respuesta `OK|SUBSCRIBED`; los números `Seq` y `Ack` avanzan con esos bytes. Las líneas rojas son retransmisiones del FIN de una conexión anterior que el servidor ya no reconocía.

La telemetría, en cambio, viaja en datagramas UDP al puerto 5001: no hay paquetes de establecimiento ni de acuse, cada datagrama tiene solo cuatro campos de encabezado (puertos, longitud y suma de comprobación) y el texto `TELEMETRY|...` se lee directamente en su carga útil. Esa diferencia es la que se ve entre las dos filas de la Tabla 7.1.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.1.10	100.58.193.142	TCP	66	42074 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=63 Len=0 TSval=5800056441 TSecr=1899272815
2	0.292680050	192.168.1.10	100.58.193.142	TCP	66	[TCP Retransmission] 42074 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=63 Len=0 TSval=5800056441 TSecr=1899272815
3	0.544997969	192.168.1.10	100.58.193.142	TCP	66	[TCP Retransmission] 42074 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=63 Len=0 TSval=5800056441 TSecr=1899272815
4	1.128783177	192.168.1.10	100.58.193.142	TCP	66	[TCP Retransmission] 42074 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=63 Len=0 TSval=5800056441 TSecr=1899272815
5	2.208992338	192.168.1.10	100.58.193.142	TCP	66	[TCP Retransmission] 42074 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=63 Len=0 TSval=5800056441 TSecr=1899272815
6	2.472947529	192.168.1.10	100.58.193.142	TCP	74	44484 → 5000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2327233821 TSecr=0 WS=1024
7	2.536338934	100.58.193.142	192.168.1.10	TCP	74	5000 → 44484 [SYN, ACK] Seq=9 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=299024445 TSecr=2327233821 WS=128
8	2.536375584	192.168.1.10	100.58.193.142	TCP	66	44484 → 5000 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2327233884 TSecr=299024445
9	2.536488943	192.168.1.10	100.58.193.142	TCP	76	44484 → 5000 [PSH, ACK] Seq=1 Ack=1 Win=64512 Len=10 TSval=2327233884 TSecr=299024445
10	2.599747698	100.58.193.142	192.168.1.10	TCP	66	5000 → 44484 [ACK] Seq=11 Ack=11 Win=65152 Len=0 TSval=299024508 TSecr=2327233884
11	2.599825408	100.58.193.142	192.168.1.10	TCP	89	5000 → 44484 [PSH, ACK] Seq=1 Ack=11 Win=65152 Len=14 TSval=299024508 TSecr=2327233884
12	2.599847988	192.168.1.10	100.58.193.142	TCP	66	44484 → 5000 [ACK] Seq=11 Ack=15 Win=64512 Len=0 TSval=2327233948 TSecr=299024508
13	4.394554552	192.168.1.10	100.58.193.142	TCP	66	[TCP Retransmission] 42074 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=63 Len=0 TSval=5800056441 TSecr=1899272815
14	4.415636350	192.168.1.10	100.58.193.142	TCP	76	44484 → 5000 [PSH, ACK] Seq=11 Ack=15 Win=64512 Len=10 TSval=232723574 TSecr=299024508
15	4.479327629	100.58.193.142	192.168.1.10	TCP	72	5000 → 44484 [PSH, ACK] Seq=15 Ack=21 Win=65152 Len=6 TSval=299026387 TSecr=232723574
16	4.479358879	192.168.1.10	100.58.193.142	TCP	66	44484 → 5000 [ACK] Seq=21 Ack=21 Win=64512 Len=0 TSval=2327235827 TSecr=299026387
17	4.542928370	100.58.193.142	192.168.1.10	TCP	254	5000 → 44484 [PSH, ACK] Seq=21 Ack=21 Win=65152 Len=180 TSval=299026451 TSecr=2327235827
18	4.542964190	192.168.1.10	100.58.193.142	TCP	66	44484 → 5000 [ACK] Seq=21 Ack=209 Win=64512 Len=0 TSval=2327235891 TSecr=299026451
19	9.926998139	192.168.1.10	100.58.193.142	TCP	66	[TCP Retransmission] 42074 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=63 Len=0 TSval=5800056441 TSecr=1899272815
20	10.030947605	192.168.1.10	100.58.193.142	TCP	77	44484 → 5000 [PSH, ACK] Seq=21 Ack=209 Win=64512 Len=11 TSval=2327241307 TSecr=299026451
21	10.102367627	100.58.193.142	192.168.1.10	TCP	71	5000 → 44484 [PSH, ACK] Seq=209 Ack=32 Win=65152 Len=5 TSval=299032011 TSecr=2327241307
22	10.102393226	192.168.1.10	100.58.193.142	TCP	66	44484 → 5000 [ACK] Seq=32 Ack=214 Win=64512 Len=0 TSval=2327241450 TSecr=299032011
23	10.106110185	100.58.193.142	192.168.1.10	TCP	104	5000 → 44484 [PSH, ACK] Seq=214 Ack=32 Win=65152 Len=30 TSval=299032074 TSecr=2327241450
24	10.106114605	192.168.1.10	100.58.193.142	TCP	66	44484 → 5000 [ACK] Seq=32 Ack=252 Win=64512 Len=0 TSval=2327241514 TSecr=299032074
25	17.632558076	192.168.1.10	100.58.193.142	TCP	66	[TCP Retransmission] 42074 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=63 Len=0 TSval=5800056441 TSecr=1899272815
26	35.041843290	192.168.1.10	100.58.193.142	TCP	66	[TCP Retransmission] 42074 → 5000 [FIN, ACK] Seq=1 Ack=1 Win=63 Len=0 TSval=5800056441 TSecr=1899272815

Figura 7.1: Captura de la conexión TCP del operador: (paquetes 6 a 8) y segmentos con mensajes TELEP.

Cuadro 7.1: Qué se observa en cada capa.

Capa	TCP (consultas)	UDP (telemetría)
Aplicación	texto TELEP (SUBSCRIBE, OK ...)	texto TELEMETRY ...
Transporte	puertos 44484 y 5000, Seq/Ack, SYN/PSH/FIN	puertos y 5001, sin conexión ni acuses
Red	192.168.1.10 → 100.58.193.142	192.168.1.10 → 100.58.193.142

8 | Pruebas y resultados

Las pruebas se organizan en tres niveles: una suite funcional local (`tests/run.py`, 14 casos), un ensayo integral con cinco nodos y dos operadores reales (`tests/end-to-end.py`) y las pruebas contra el servidor desplegado en la nube. La última ejecución local de la suite, del 20 de septiembre de 2026, aprobó los 14 casos (`tests/results/functional.json`); cubren HTTP, sesiones y estadísticas, errores, alertas, dos operadores simultáneos, suscriptor lento, recuperación y el controlador de la GUI.

Cuadro 8.1: Demostraciones que pide el enunciado y cómo se realizaron (20 de septiembre de 2026).

Requisito	Cómo se probó	Resultado observado
Varios nodos simultáneos	cinco nodos ejecutados desde los computadores del equipo contra la nube	los cinco aparecen activos en la página de estado y en <code>GET_NODES</code> (Figuras A.6 y A.12)
Comunicación TCP	operador conectado por el nombre DNS; <code>GET_STATUS</code> y <code>GET_NODES</code>	respuestas OK con el estado y la lista de nodos (Figura A.7); handshake y segmentos en la Figura 7.1
Comunicación UDP	nodos enviando telemetría; <code>STATS</code> por nodo	100 datagramas enviados y 100 únicos recibidos por nodo (Tabla 8.2); datagramas visibles en la Figura A.14
Consultas de operadores	dos operadores conectados al mismo tiempo	ambos reciben sus respuestas sin bloquearse (Figuras A.8 y A.11)
Generación de alerta	nodo con temperatura forzada a 45	alerta <code>TEMP_HIGH</code> registrada y visible en la página de estado (Figura A.6)
Funcionamiento concurrente	cinco nodos y dos operadores a la vez sobre el mismo servidor	telemetría, consultas y alerta atendidas simultáneamente (mismas figuras anteriores)
Acceso por DNS	<code>nslookup</code> y clientes con <code>--host telemetria-eafit.duckdns.org</code>	el nombre resolvió a 100.58.193.142 ese día (Figura A.5); ningún cliente usa una IP
Funcionamiento desde Internet	clientes desde los tres computadores del equipo, en redes distintas a la de la instancia	conexión, telemetría y consultas correctas desde los tres (Anexo A)
Recuperación ante desconexión o mensaje incorrecto	cuatro mensajes inválidos por TCP y tres datagramas basura por UDP contra el servidor del commit final	respuestas <code>ERR 100</code> , <code>ERR 102</code> , <code>ERR 100</code> y <code>ERR 103</code> , y la misma conexión siguió respondiendo OK; los datagramas se descartaron (<code>udp_invalid=3</code>) sin afectar el servicio (Figura 8.1)

```

> this is not TELEP
< ERR|100|BAD_FORMAT
> GET_LAST|NOPE
< ERR|102|UNKNOWN_NODE
> HELLO|bad id
< ERR|100|BAD_FORMAT
> XXXXXXXX... (1100 bytes)
< ERR|103|TOO_LONG
> GET_STATUS
< OK|boot_id=1789964440044941389;uptime=0;registered=0;active=0;tcp_ok=1;udp_ok=1;...
> BYE
< OK|BYE

(UDP) garbage / TELEMETRY|incomplete / bytes binarios -> descartados
> GET_STATUS
< OK|...;udp_rx=0;udp_invalid=3;...

```

Figura 8.1: Prueba de recuperación sobre el servidor del commit final: cada mensaje inválido recibe un ERR y la conexión sigue abierta; los datagramas inválidos se cuentan en `udp_invalid` y el servidor continúa atendiendo.

8.1 | Mensajes transmitidos, recibidos y perdidos

Cada nodo cuenta cuántos datagramas intentó enviar y, al terminar, se lo informa al servidor con `REPORT`. El servidor, por su lado, marca en un mapa de bits qué números de secuencia recibió y con `STATS` devuelve cuántos únicos llegaron, cuántos duplicados o reordenados vio, y la pérdida estimada (enviados menos únicos recibidos). Para comprobar que la cuenta es correcta usamos la opción `--drop`, que omite un

porcentaje de intentos antes de enviarlos; esas omisiones se informan aparte y no se confunden con pérdida de red.

La Tabla 8.2 muestra la prueba hecha contra el servidor en la nube el 20 de septiembre con dos nodos de 100 intentos cada uno, a 20 datagramas por segundo. El nodo con `--drop 25` omitió exactamente 25, envió 75 y el servidor recibió los 75; el nodo sin omisiones envió 100 y llegaron 100. En ninguno de los dos hubo pérdida en la red.

Cuadro 8.2: Contadores del nodo y del servidor en la prueba contra la nube (100 intentos por nodo).

Nodo	Intentos	Enviados	Omitidos	Únicos recibidos	Duplicados	Reordenados	Pérdida
INF-PLAIN	100	100	0	100	0	0	0
INF-DROP (<code>--drop 25</code>)	100	75	25	75	0	0	0

Sobre un periodo más largo sí aparece pérdida real: el resumen `GET.STATUS` del servidor, tras 2 horas con los nodos del equipo enviando desde varias redes, registraba 18 999 datagramas enviados, 19 005 únicos recibidos (incluye una sesión que reportó antes de terminar) y una pérdida estimada de 1. Es el comportamiento esperado de UDP en Internet y la razón por la que la telemetría, y no las consultas ni las alertas, va por ese transporte.

9 | Problemas encontrados y soluciones implementadas

Alertas repetidas. Al principio el servidor generaba una alerta por cada medición mientras el valor siguiera alto, así que un nodo con la temperatura en 45 producía una alerta por segundo. Ahora compara con la medición anterior y solo alerta cuando el valor cruza el umbral.

Pérdidas mal contadas. Contábamos como perdidos los huecos en el número de secuencia, pero al reiniciar un nodo la secuencia volvía a cero y el servidor contaba miles de pérdidas falsas. Lo resolvimos con una sesión por ejecución del nodo y con el mensaje `REPORT`, en el que el nodo dice cuántos datagramas envió.

Alertas mezcladas con respuestas. Cuando el servidor empezó a enviar alertas a los operadores, una alerta podía salir en medio de una respuesta de varias líneas. Ahora cada conexión tiene un solo hilo que escribe y las alertas esperan en una cola hasta que la respuesta termina.

La regla UDP en AWS. La consola crea las reglas como TCP por defecto. Con el puerto 5001 en TCP todo parecía funcionar, pero el servidor nunca recibía telemetría. Había que crear la regla como UDP.

La IP cambia en cada sesión del laboratorio. AWS Academy apaga la instancia a las cuatro horas y al encenderla le asigna otra IP, lo que dejaba el nombre DNS apuntando a una dirección vieja. Pusimos un servicio en la instancia que actualiza DuckDNS en cada arranque.

El puerto UDP en Docker. Publicar 5001:5001 sin `/udp` deja el contenedor funcionando y la web respondiendo, pero ningún datagrama entra. Se agregó el sufijo en `docker-compose.yml`.

La imagen no compilaba. Al instalar solo `gcc` faltaban las cabeceras de C estándar. Se agregó `libc6-dev` al `Dockerfile`.

Nodos que morían sin reportar. Al detenerlos desde un script no recibían `Ctrl-C` y terminaban sin enviar su reporte. El nodo ahora también atiende `SIGTERM`.

10 | Conclusiones

Diseñar nuestro propio protocolo nos obligó a decidir cosas que con HTTP o MQTT vienen resueltas: cómo se separan los campos, cuánto puede medir un mensaje, qué responde el servidor ante cada error y qué hace con un mensaje que no entiende. Esas decisiones son las que después se ven una a una en Wireshark.

La elección entre UDP y TCP dejó de ser teórica al ver las capturas: un datagrama de telemetría es un solo paquete sin conexión previa, mientras que una consulta TCP necesita el handshake, los acuses y el

cierre. Para mediciones que se repiten cada segundo lo primero alcanza; para una alerta o una consulta que no pueden perderse, lo segundo es necesario.

En la nube aprendimos que “funciona en localhost” no dice nada sobre “funciona desde Internet”: el grupo de seguridad distingue TCP de UDP, Docker necesita publicar el puerto con el protocolo correcto y la IP de la instancia cambia en cada sesión del laboratorio. Resolver el servidor por un nombre DNS que se actualiza solo fue lo que permitió que los clientes nunca tuvieran que cambiar.

Por último, contar mensajes perdidos en UDP solo tiene sentido si se define bien qué se cuenta: tuvimos que separar los intentos, los envíos, las omisiones simuladas y los duplicados para que el número final signifique algo.

A | Evidencias individuales

Cada integrante ejecutó un componente del sistema desde su propio computador, conectándose al servidor desplegado en la nube. En cada captura se ven, en la misma terminal, la salida de `whoami`, `hostname` y `date` seguida de la ejecución del componente.

A.1 | Juan Andrés Young

Componente: servidor en C, despliegue en AWS con Docker y DNS.

Contribución: escribí buena parte del servidor en C y la estructura inicial del repositorio, configuré la instancia EC2 y su grupo de seguridad, y monté el despliegue con Docker y el nombre en DuckDNS con el servicio que lo actualiza en cada arranque.

Evidencias:

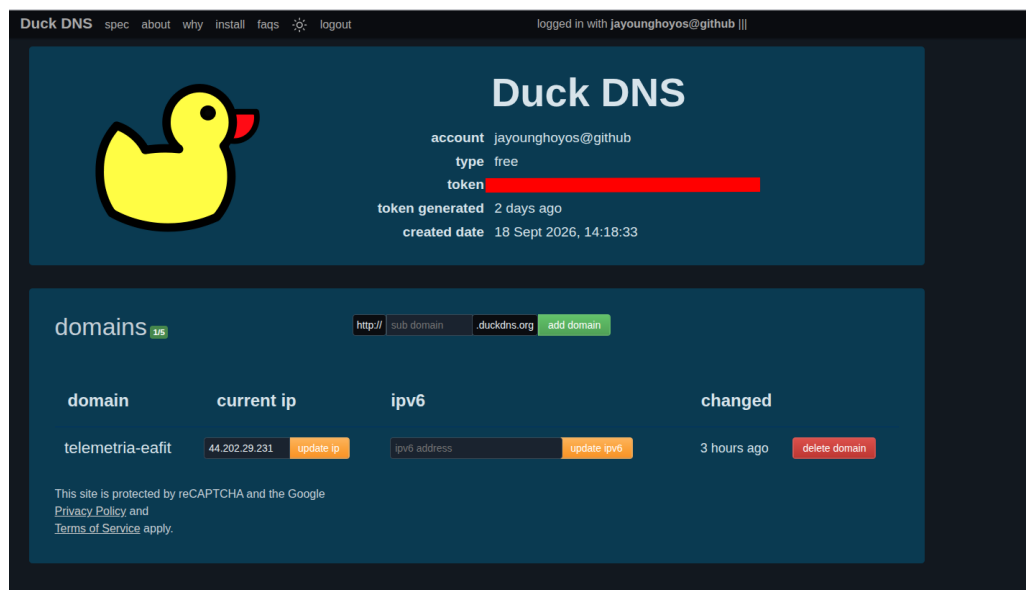
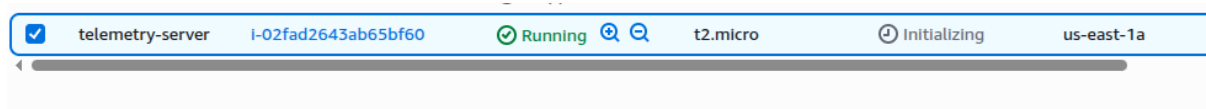


Figura A.1: Registro `telemetria-eafit` en DuckDNS apuntando a la IP pública de la instancia; el token se muestra oculto.



i-02fad2643ab65bf60 (telemetry-server)

Name	Security group rule ID	Port range	Protocol	Source
-	sgr-0b71159894254c96b	8080	TCP	0.0.0.0/0
-	sgr-0709a329fc490aeda	5000	TCP	0.0.0.0/0
-	sgr-065ed62ab34540720	22	TCP	0.0.0.0/0
-	sgr-073c40c6befd9fdce	5001	UDP	0.0.0.0/0

Figura A.2: Instancia `telemetry-server` en ejecución en la consola de EC2 (`t2.micro`, `us-east-1a`) y las cuatro reglas de entrada de su grupo de seguridad: 22/TCP, 5000/TCP, 5001/UDP y 8080/TCP.



Figura A.3: Identificación del equipo (`whoami`, `hostname`, `date`) y nodo `NODE06` ejecutándose contra el servidor en la nube: resuelve el nombre DNS, se registra y envía sus datagramas; al terminar reporta 10 intentos y 10 enviados.

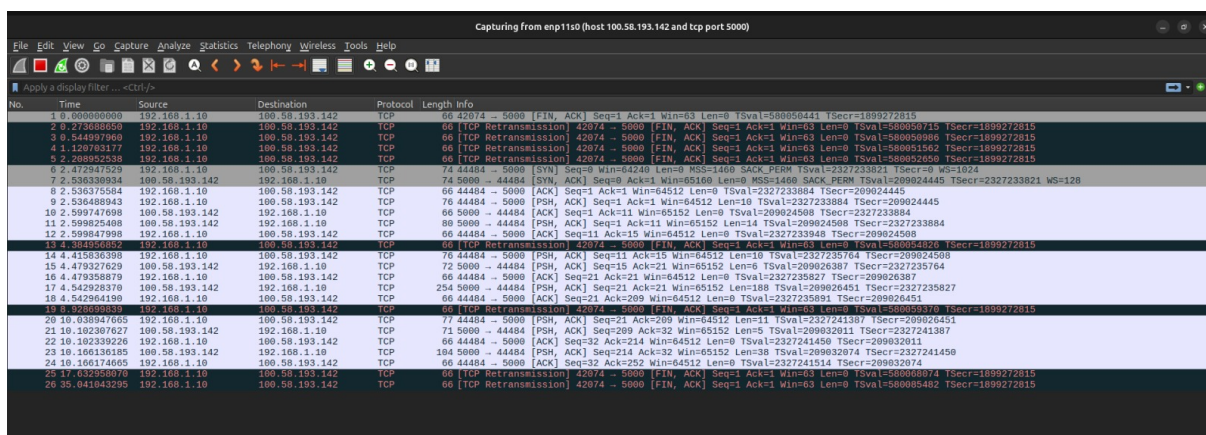
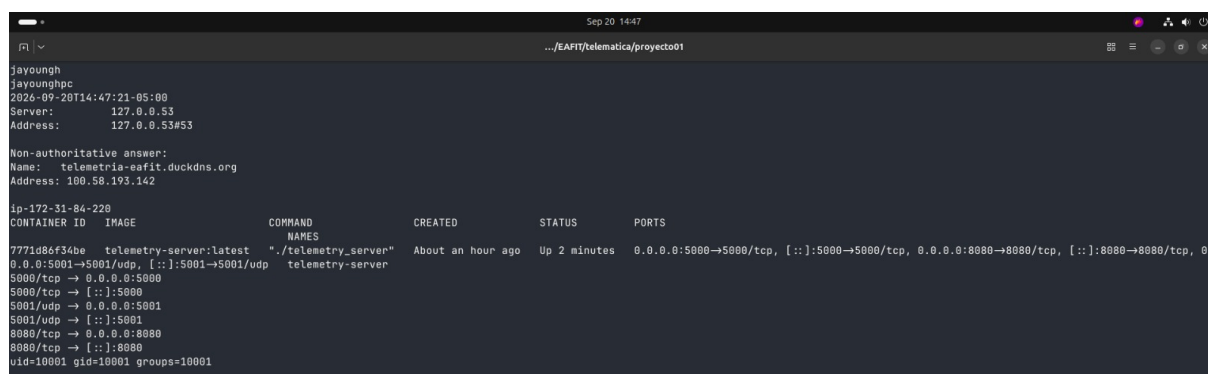


Figura A.4: Captura de Wireshark de la conexión TCP del operador al puerto 5000 del servidor en la nube: handshake, segmentos con mensajes TELEP y retransmisiones de una conexión anterior.



```
Sep 20 14:47
.../EAFIT/telematica/proyecto01

jyoung@hpc
jyoung@hpc
2026-09-20T14:47:21-05:00
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   telemetria-eafit.duckdns.org
Address: 100.58.193.142

ip-172-31-84-220
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
7771d86f34be   telemetry-server:latest             "/telemetry_server"      About an hour ago Up 2 minutes   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:5001->5001/udp, [::]:5001->5001/udp
5000/tcp -> 0.0.0.0:5000
5000/tcp -> [::]:5000
5001/udp -> 0.0.0.0:5001
5001/udp -> [::]:5001
8080/tcp -> 0.0.0.0:8080
8080/tcp -> [::]:8080
uid=10001 gid=10001 groups=10001
```

Figura A.5: Identificación del equipo, resolución DNS de `telemetria-eafit.duckdns.org` con `nslookup`, y contenedor `telemetry-server` en ejecución en la instancia con los puertos 5000/TCP, 5001/UDP y 8080/TCP publicados y usuario sin privilegios (`uid=10001`).

A.2 | Samuel López

Componente: validación funcional de TELEP/2.0 y del cliente operador.

Contribución: probé consultas, dos operadores simultáneos y la generación de alertas contra el servidor en la nube; registré evidencias de su funcionamiento.

Evidencias:

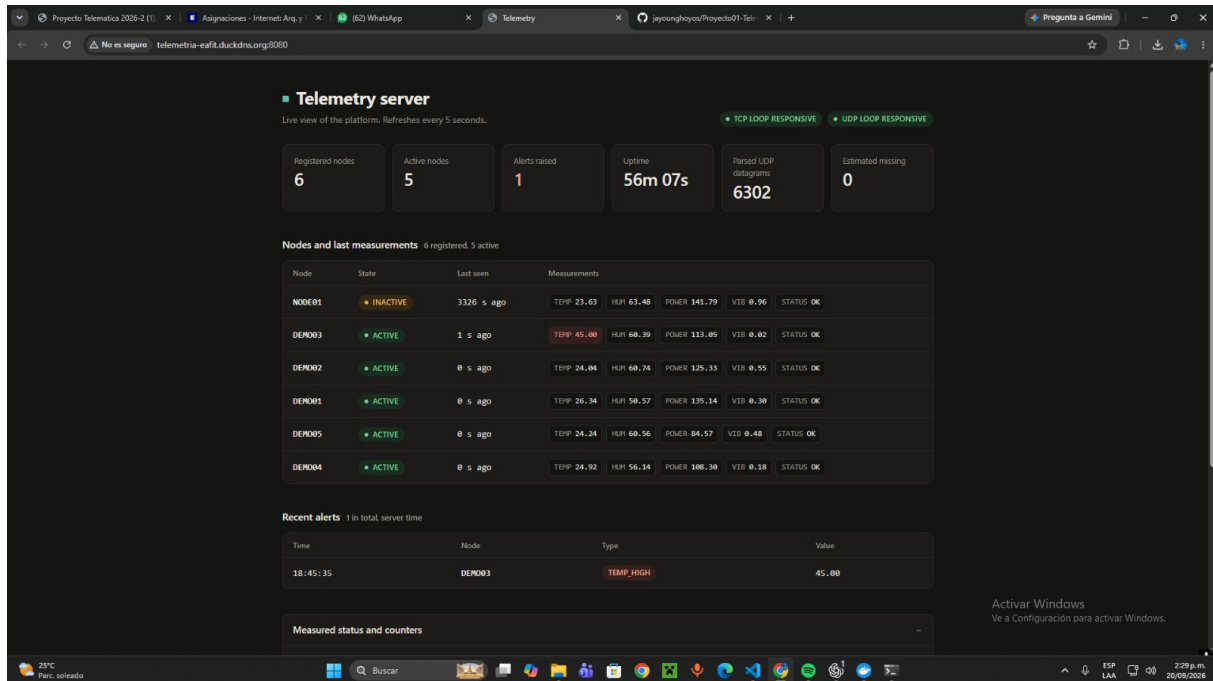


Figura A.6: Página de estado del servidor en la nube con cinco nodos activos, mediciones y una alerta de temperatura.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\USUARIO> cd 'C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura'
PS C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura> whoami
desktop-Scion36\usuario
PS C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura> hostname
desktop-Scion36
PS C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura> Get-Date -Format o
2626-09-20T14:28:02.3301964-05:00
PS C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura> git rev-parse HEAD
b8d99dc80e9d90699808f0f527c76075a3308
PS C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura> Resolve-DnsName telemetria-eafit.duckdns.org

Name                                     Type      TTL      Section  IPAddress
----
telemetria-eafit.duckdns.org            A         55       Answer   100.58.193.142

PS C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura> Test-NetConnection telemetria-eafit.duckdns.org -Port 5000

ComputerName      : telemetria-eafit.duckdns.org
RemoteAddress     : 100.58.193.142
RemotePort        : 5000
InterfaceAlias    : Ethernet 2
SourceAddress     : 192.168.78.172
TcpTestSucceeded  : True

PS C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura> python .\client\operator_cli.py --host telemetria-eafit.duckdns.org
connected to telemetria-eafit.duckdns.org (100.58.193.142:5000)
subscribed: new alerts arrive on this connection (ALERT)...

=== TELEP/2.0 operator ===
1) Registered / active nodes      (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node                 (GET_LAST|<id>)
4) Alerts                         (GET_ALERTS)
5) System status                  (GET_STATUS)
6) Send a raw command
8) Quit                           (BYE)

option: 5
boot_id      1789929287928376140
uptime      3402
registered   6
active       5
tcp_ok       1
udp_ok       1
udp_rx       6382
udp_invalid  0
udp_unknown  0
attempts     6372
sent         6372
omitted      0
send_errors  0
unique       6382
duplicates   0
reordered    0
loss_estimated 0
reports      6
final_reports 1
alerts       1
slow_disconnected 0

=== TELEP/2.0 operator ===
1) Registered / active nodes      (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node                 (GET_LAST|<id>)
4) Alerts                         (GET_ALERTS)
5) System status                  (GET_STATUS)
6) Send a raw command
8) Quit                           (BYE)

option: 1
6 registered nodes
NODE01      INACTIVE last data 3368 s ago
DEMO03      ACTIVE   last data 0 s ago
DEMO02      ACTIVE   last data 0 s ago
DEMO01      ACTIVE   last data 0 s ago
DEMO05      ACTIVE   last data 0 s ago
DEMO04      ACTIVE   last data 0 s ago
  
```

Figura A.7: Conexión al servidor mediante DNS y ejecución de las consultas GET_STATUS y GET_NODES.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\USUARIO> cd 'C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura'
PS C:\Users\USUARIO\OneDrive\Desktop\Telematica\Proyecto01-Telemetria-Gestion-de-Infraestructura> python .\client\operator_cli.py --host telemetria-eafit.duckdns.org
connected to telemetria-eafit.duckdns.org (100.58.193.142:5000)
subscribed: new alerts arrive on this connection (ALERT)...

=== TELEP/2.0 operator ===
1) Registered / active nodes      (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node                 (GET_LAST[<id>])
4) Alerts                         (GET_ALERTS)
5) System status                  (GET_STATUS)
6) Send a raw command             (BYE)
0) Quit

option: 5
boot_id      1789929207928376140
uptime      4083
registered   7
active       5
tcp_ok       1
udp_ok       1
udp_rx       8012
udp_invalid  0
udp_unknown  0
attempts     8002
sent         8002
omitted      0
send_errors  0
unique       8012
duplicates   0
reordered    0
loss_estimated 0
reports      7
final_reports 2
alerts       1
slow_disconnected 0

=== TELEP/2.0 operator ===
1) Registered / active nodes      (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node                 (GET_LAST[<id>])
4) Alerts                         (GET_ALERTS)
5) System status                  (GET_STATUS)
6) Send a raw command             (BYE)
0) Quit

option:

7 registered nodes
NODE01  INACTIVE  last data 4040 s ago
DEN003  ACTIVE    last data 1 s ago
DEN002  ACTIVE    last data 1 s ago
DEN001  ACTIVE    last data 1 s ago
DEN005  ACTIVE    last data 1 s ago
DEN004  ACTIVE    last data 1 s ago
NODEJUAN INACTIVE  last data 651 s ago

=== TELEP/2.0 operator ===
1) Registered / active nodes      (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node                 (GET_LAST[<id>])
4) Alerts                         (GET_ALERTS)
5) System status                  (GET_STATUS)
6) Send a raw command             (BYE)
0) Quit

option:
  
```

Figura A.8: Prueba de concurrencia con dos clientes operadores conectados simultáneamente al servidor.

A.3 | Ismael García

Componente: página de estado HTTP del servidor, ejecución de nodos y operadores contra el servidor en la nube, y captura de tráfico con Wireshark.

Contribución: rediseñó la página de estado en los archivos `server/http_status_server.c` y `server/status_page.css.h`. El desarrollo se integró mediante la rama `ui/status-page`, en el commit `cf2e5ec`, y posteriormente fue fusionado en `main`. También ejecutó nodos y dos operadores simultáneos contra `telemetria-eafit.duckdns.org`, y realizó la captura y el análisis de tráfico TCP, UDP y DNS.

Evidencias:

```

PS C:\Users\Ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> whoami
desktop-3fed0e\ismael
PS C:\Users\Ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> hostname
DESKTOP-3FED0E
PS C:\Users\Ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> Get-Date -Format o
2026-09-20T15:00:00.3739876-05:00
PS C:\Users\Ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> git rev-parse HEAD
b8a944dc08e48d90d9598f8f527c76075a35b0
PS C:\Users\Ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> Resolve-DnsName telemetria-eafit.duckdns.org -Server 8.8.8.8

Name                               Type      TTL      Section  IPAddress
----                               -
telemetria-eafit.duckdns.org        A         60      Answer   100.58.193.142

PS C:\Users\Ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> Test-NetConnection telemetria-eafit.duckdns.org -Port 5000

ComputerName      : telemetria-eafit.duckdns.org
RemoteAddress     : 100.58.193.142
RemotePort        : 5000
InterfaceAlias     : Ethernet
SourceAddress     : 192.168.40.50
TcpTestSucceeded  : True

PS C:\Users\Ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> python .\client\ode.py --id MODE11 --host telemetria-eafit.duckdns.org --count 10
{"event": "control", "host": "telemetria-eafit.duckdns.org", "resolved": "100.58.193.142", "session": "bea2c328ec314e94b273d35cd352d587", "attempts": 0, "sent": 0, "omitted": 0, "send_errors": 0}
{"event": "control", "host": "telemetria-eafit.duckdns.org", "resolved": "100.58.193.142", "session": "bea2c328ec314e94b273d35cd352d587", "attempts": 3, "sent": 3, "omitted": 0, "send_errors": 0}
{"event": "control", "host": "telemetria-eafit.duckdns.org", "resolved": "100.58.193.142", "session": "bea2c328ec314e94b273d35cd352d587", "attempts": 6, "sent": 6, "omitted": 0, "send_errors": 0}
{"event": "control", "host": "telemetria-eafit.duckdns.org", "resolved": "100.58.193.142", "session": "bea2c328ec314e94b273d35cd352d587", "attempts": 9, "sent": 9, "omitted": 0, "send_errors": 0}
{"event": "final", "id": "MODE11", "session": "bea2c328ec314e94b273d35cd352d587", "final_ack": true, "attempts": 10, "sent": 10, "omitted": 0, "send_errors": 0}

```

Figura A.9: Evidencia individual de Ismael García con la identificación del equipo, fecha, resolución DNS, prueba de conexión al puerto TCP 5000 y ejecución contra el servidor en la nube.

```

PS C:\Users\Ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> python .\client\operator_cli.py --host telemetria-eafit.duckdns.org
Connected to telemetria-eafit.duckdns.org (100.58.193.142:5000)
Subscribed: new alerts arrive on this connection (ALERT...)

--- TELNET/2.0 operator ---
1) Registered / Active nodes (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node (GET_LASTID)
4) Alerts (GET_ALERTS)
5) System status (GET_STATUS)
6) Send a raw command
7) Quit (BYE)

option: 1
9 registered nodes
NodeID    ACTIVE    last data 1 s ago
DEP001    ACTIVE    last data 2 s ago
DEP002    ACTIVE    last data 2 s ago
DEP003    ACTIVE    last data 2 s ago
DEP004    ACTIVE    last data 2 s ago
NOD005    INACTIVE  last data 923 s ago
SOP006    INACTIVE  last data 626 s ago
NOD007    INACTIVE  last data 543 s ago
NOD011    INACTIVE  last data 76 s ago

--- TELNET/2.0 operator ---
1) Registered / Active nodes (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node (GET_LASTID)
4) Alerts (GET_ALERTS)
5) System status (GET_STATUS)
6) Send a raw command
7) Quit (BYE)

option: 3
NodeID: NOD011
NOD011    15:00:41 TBPW-35.82 H9N-58.17 P0NE8-133.65 VEB-94 STATUS-GOK

--- TELNET/2.0 operator ---
1) Registered / Active nodes (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node (GET_LASTID)
4) Alerts (GET_ALERTS)
5) System status (GET_STATUS)
6) Send a raw command
7) Quit (BYE)

option: 5
NodeID    1709931038223821793
uptime    1987
registered 9
active    5
tx_ok     1
udp_ok    1
udp_rx    2523
udp_inval 0
udp_timeout 18
sent       2498
attempts   0
omitted    0
send_errors 0
unique     2513
duplicates 0
reordered  0
lost_oldest 0
reports    9
final_reports 4
alerts     1
slow_disconnect 0

--- TELNET/2.0 operator ---
1) Registered / Active nodes (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node (GET_LASTID)
4) Alerts (GET_ALERTS)
5) System status (GET_STATUS)
6) Send a raw command
7) Quit (BYE)

```

Figura A.10: Inicio del cliente operador y conexión mediante el nombre DNS del servidor desplegado en la nube.

```

PS C:\Users\ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> whoami
desktop-3f6dode\ismael
PS C:\Users\ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> hostname
desktop-3f6dode
PS C:\Users\ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> Get-Date -Format o
2026-09-20T15:15:38.7272245-05:00
PS C:\Users\ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> python .\client\operator_cli.py --
host telemetria-eafit.duckdns.org
connected to telemetria-eafit.duckdns.org (100.58.193.142:5900)
subscribed: new alerts arrive on this connection (ALERT)...

=== TELEP/2.0 operator ===
1) Registered / active nodes      (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node                 (GET_LAST|id)
4) Alerts                         (GET_ALERTS)
5) System status                  (GET_STATUS)
6) Send a raw command
0) Quit                           (BYE)

option: 1
11 registered nodes
DEN001 INACTIVE last data 348 s ago
DEN002 INACTIVE last data 348 s ago
DEN005 INACTIVE last data 348 s ago
DEN001 INACTIVE last data 348 s ago
DEN003 INACTIVE last data 348 s ago
NODE06 INACTIVE last data 1768 s ago
SAN0020 INACTIVE last data 1409 s ago
NODE05 INACTIVE last data 1400 s ago
NODE11 INACTIVE last data 939 s ago
SAN0021 INACTIVE last data 684 s ago
SAN0022 INACTIVE last data 635 s ago

=== TELEP/2.0 operator ===
1) Registered / active nodes      (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node                 (GET_LAST|id)
4) Alerts                         (GET_ALERTS)
5) System status                  (GET_STATUS)
6) Send a raw command
0) Quit                           (BYE)

option:

PS C:\Users\ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> whoami
desktop-3f6dode\ismael
PS C:\Users\ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> hostname
desktop-3f6dode
PS C:\Users\ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> Get-Date -Format o
2026-09-20T15:15:39.9903735-05:00
PS C:\Users\ismael\Documents\Repositories\Proyecto01-Telemetria-Gestion-de-Infraestructura> python .\client\operator_cli.py --
host telemetria-eafit.duckdns.org
connected to telemetria-eafit.duckdns.org (100.58.193.142:5900)
subscribed: new alerts arrive on this connection (ALERT)...

=== TELEP/2.0 operator ===
1) Registered / active nodes      (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node                 (GET_LAST|id)
4) Alerts                         (GET_ALERTS)
5) System status                  (GET_STATUS)
6) Send a raw command
0) Quit                           (BYE)

option: 5
boot_id 1789933510221881765
uptime 1074
registered 11
active 0
top_ok 1
udp_ok 1
udp_rx 3073
udp_invalid 0
udp_unknown 10
attempts 3658
sent 3658
omitted 0
send_errors 0
unique 3663
duplicates 0
reordered 0
loss_estimated 0
reports 11
final_reports 6
alerts 1
slow_disconnected 0

=== TELEP/2.0 operator ===
1) Registered / active nodes      (GET_NODES)
2) Last measurements of all nodes (GET_NODES + GET_LAST)
3) Query one node                 (GET_LAST|id)
4) Alerts                         (GET_ALERTS)
5) System status                  (GET_STATUS)
6) Send a raw command
0) Quit                           (BYE)

option:

```

Figura A.11: Prueba de concurrencia con dos clientes operadores conectados simultáneamente al servidor en la nube.

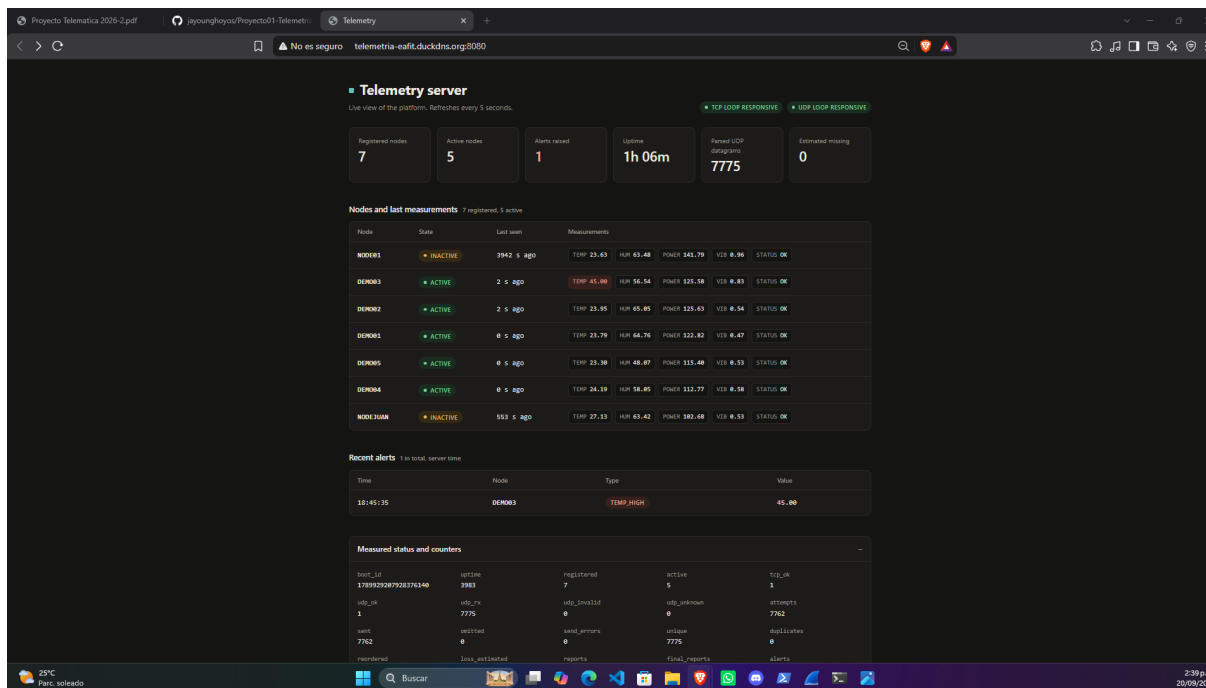


Figura A.12: Página de estado HTTP con los nodos registrados, las mediciones recientes, las alertas y los contadores del servidor.

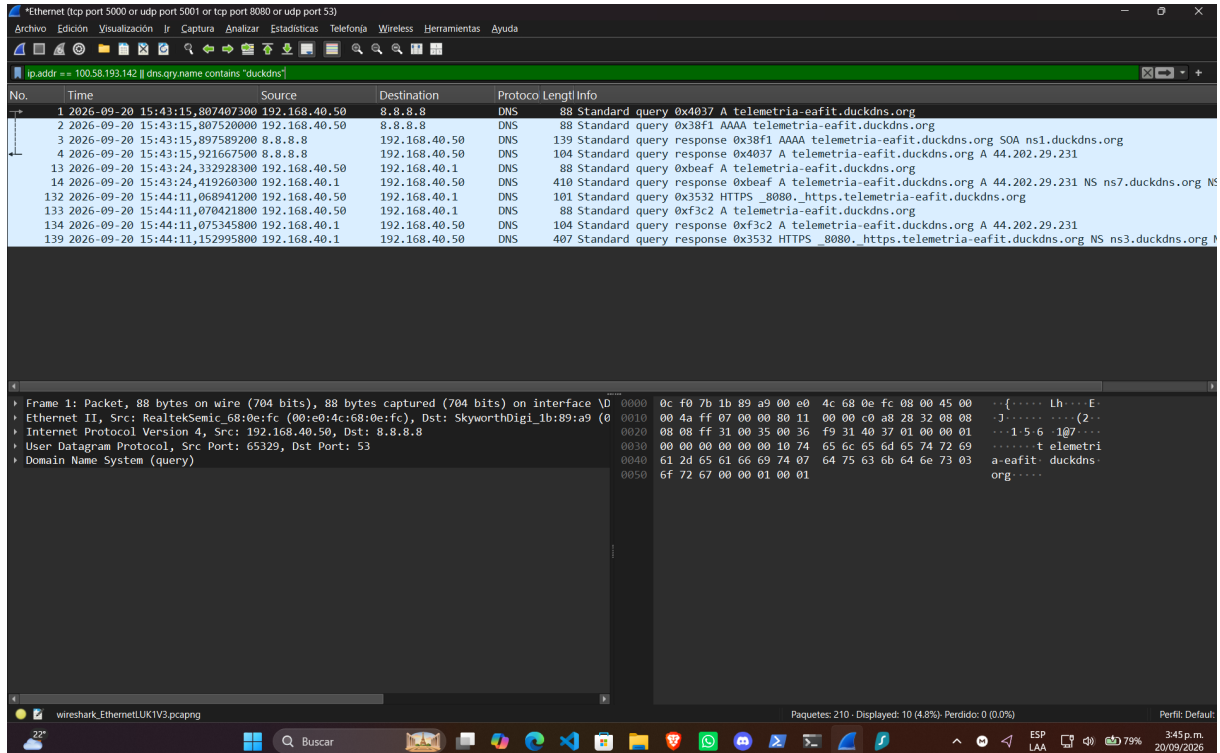


Figura A.13: Configuración del filtro de captura de Wireshark para registrar el tráfico TCP del puerto 5000 y UDP del puerto 5001.

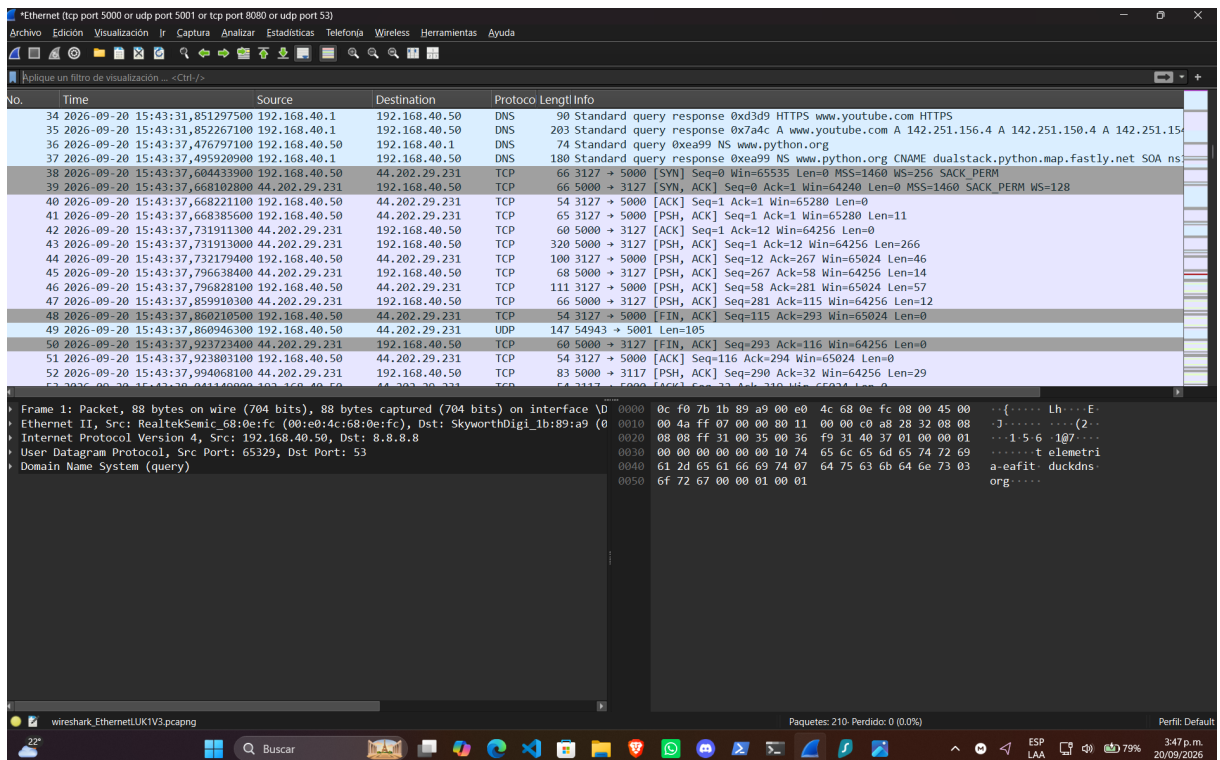


Figura A.14: Tráfico TCP y UDP capturado con Wireshark durante la comunicación entre los clientes y el servidor en la nube.

B | Video de sustentación

Enlace: <https://youtu.be/ng1NRy6YbEc>.

El video muestra la arquitectura, el protocolo, el servidor en la nube y Docker en ejecución, cinco nodos simultáneos, el cliente operador, la comunicación TCP y UDP, la generación de una alerta, una captura de Wireshark y los resultados; cada integrante presenta la parte que desarrolló.

C | Referencias y archivos

- Repositorio: <https://github.com/jayounghoyos/Proyecto01-Telemetria-Gestion-de-Infraestructura>.
- Especificación del protocolo: docs/PROTOCOL.md.
- Arquitectura: docs/ARCHITECTURE.md.
- Despliegue: docs/DEPLOY.md.
- Fuentes de los diagramas: docs/diagrams/*.pen.
- Enunciado: docs/Proyecto Telematica 2026-2.pdf.